Thunderbolt: Fast Asynchronous Off-Chain Bitcoin Transfers

Anonymous Author(s)

Abstract

We present Thunderbolt, an off-chain protocol that transfers Bitcoin UTXO ownership in ≈**1 s**, requires **no channel graph, no routing, and no liquidity rebalancing**, and lets the **recipient be offline** at the time of transfer. A single UTXO is locked once on-chain under a fixed public key; ownership then passes through an unbounded sequence of holders; each transfer is a purely off-chain, asynchronous operation whose on-chain cost is zero. The chain sees exactly two transactions regardless of how many transfers occur.

The cryptographic core is a pair of Sigma-protocol NIZK proofs, the *Invoice Consistency Proof* and the *Transfer Consistency Proof*, which jointly enforce a "one add, one subtract" invariant on an additively shared key. Both instantiate Camenisch–Shoup verifiable encryption of discrete logarithms without trusted setup or circuit-level tooling. When the committee is a threshold group, Simple VESS (Verifiable Encrypted Secret Sharing) distributes the committee share under a standard (t, n)-honesty assumption, the only trust requirement Thunderbolt introduces beyond the base chain; routing and liquidity constraints are absent by construction.

On a commodity laptop, a complete off-chain transfer finishes in **1.05 s** with a combined proof size of **3.8 KB**. Against SNARK backends on the same relations, our proofs are **100–1000× faster**, as large-modulus Paillier arithmetic over $\mathbb{Z}_{N^2}$ requires a disproportionately large number of constraints in circuit-based proving systems.

CCS Concepts

• **Security and privacy** → **Cryptography**; **Distributed systems security**.

Keywords

Bitcoin, Off-Chain Transfers, Verifiable Encryption, Paillier Cryptosystem, Zero-Knowledge Proofs, Payment Channels

ACM Reference Format:

Anonymous Author(s). 2026. Thunderbolt: Fast Asynchronous Off-Chain Bitcoin Transfers. In *Proceedings of Proceedings of the 2026 ACM Conference on Computer and Communications Security (CCS '26).* ACM, New York, NY, USA, 14 pages. https://doi.org/10.1145/XXXXXXX.XXXXXXX

1 Introduction

Bitcoin [46] intentionally constrains throughput and expressiveness at the base layer: blocks are small, scripts are limited, and every state change pays the cost of global verification. This conservatism underpins Bitcoin's credibility as a settlement layer, but makes it unsuitable as a fast, low-cost medium for everyday value transfer. The dominant response has been the *payment channel*: two (or more) parties lock funds in an on-chain UTXO, exchange signed off-chain updates, and fall back to the chain only to open or close. The Lightning Network [51] scales this to a routed graph of channels and is today the most widely deployed Bitcoin layer-2.

Lightning optimizes for multi-hop routing across pre-funded, bidirectionally-online channels, leaving room for complementary protocols. Some scenarios call for *asynchronous* transfers (sender and receiver need not be online simultaneously), *route-free* operation (no capacity-sufficient path is needed), and *sequential ownership migration* (the same UTXO passes through an unbounded sequence of holders) — as in payroll batches, cold-wallet receipts, custody chains, and intermittently-online devices.

A different question. Instead of asking "how do we route a payment across a graph of pre-funded channels," we ask: *can the ownership of a single Bitcoin UTXO be transferred off-chain, asynchronously, from an arbitrary current holder to an arbitrary next recipient, without ever touching the chain and without either party being online at the same time as the other?* If the answer is yes, then the on-chain footprint of any sequence of transfers, regardless of length, remains one lock, one unlock. The chain sees only the endpoints; everything in between is off-chain state.

Thunderbolt. This paper presents Thunderbolt, an off-chain protocol that answers the above question affirmatively. In Thunderbolt, a Bitcoin UTXO is locked once, under a fixed public key $P = (x_0 + y_0)\,G$ that is additively shared between a *holder* and a *Committee*. Ownership of the UTXO is then transferred through a sequence of "one add, one subtract" updates: at each step s, a fresh refresh value r_s is added to the holder share and subtracted from the committee share, so that $x_{s+1} + y_{s+1} = x_s + y_s$ and the on-chain key P is preserved. The holder share itself lives inside a Paillier ciphertext that travels from one recipient to the next, while the committee share is maintained (or reshared) by a committee that never sees the holder's secret. A transfer from Alice to Bob requires no interaction beyond Bob posting a one-time *invoice*; neither party needs to be online simultaneously, and no route through a channel graph is needed. The invoice-and-consume interaction is *recursive*: Bob, once he has consumed Alice's transfer, can immediately serve as the next sender to Carol without any on-chain activity.

Cryptographic approach. The correctness of each transfer step is certified by two Sigma-protocol NIZK proofs, instantiating Camenisch–Shoup verifiable encryption of discrete logarithms [16]: the *Invoice Consistency Proof* binds the refresh value r_s across the elliptic-curve delta and two Paillier ciphertexts via a single shared Sigma response, and the *Transfer Consistency Proof* binds the sender's holder secret to the anchor point. The committee role can be distributed to a threshold group via Simple VESS [56]. Both primitives are well-studied; Thunderbolt contributes a new *usage*

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

CCS '26, The Hague, The Netherlands

© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 978-1-4503-XXXX-X/2026/11
https://doi.org/10.1145/XXXXXXX.XXXXXXX

scenario rather than a new assumption. We compare this approach against general-purpose SNARK backends in **??**.

Contributions. The main contributions of this paper are:

(1) We introduce Thunderbolt, an off-chain protocol that enables asynchronous, recursive transfer of Bitcoin UTXO ownership between uncoordinated users under a fixed on-chain lock key.
(2) We give two compact Sigma-protocol-based NIZK proofs (the Invoice Consistency Proof and the Transfer Consistency Proof) that jointly certify a single correct transfer step. Both are direct instantiations of Camenisch–Shoup verifiable encryption of discrete logarithms, with the Invoice proof using a single shared Sigma response to bind three invoice objects to the same refresh value r_s.
(3) We evaluate Thunderbolt on commodity hardware and compare it against Lightning and general-purpose SNARK backends, showing orders-of-magnitude lower proving cost than a circuit-based realization.

Roadmap. Section 2 recalls background primitives. Section 3 defines the system model and motivates the VEDL + VESS composition. Section 4 presents the construction, including the Invoice and Transfer Consistency Proofs and the channel lifecycle. Section 5 formalizes security via a UC ideal functionality $\mathcal{F}_{\text{Thunderbolt}}$ and analyzes attack vectors. Section 6 addresses four research questions covering single-transfer latency (RQ1), scalability (RQ2), threshold-committee performance (RQ3), and comparison with SNARK backends (RQ4). Section 7 surveys related work and Section 8 concludes.

2 Preliminaries

We write λ for the security parameter and $\mathsf{negl}(\lambda)$ for an unspecified negligible function of λ.

2.1 Elliptic-Curve Groups

Let $\mathbb{G}$ be an elliptic-curve group of prime order q. We fix two independent generators of $\mathbb{G}$: the standard secp256k1 base point G, used for the on-chain lock key $P = (x_s + y_s)\,G$, and a second nothing-up-my-sleeve (NUMS) generator H, used for all holder anchors $X_s = x_s\,H$ and invoice deltas $D_s = r_s\,H$. Independence of G and H (i.e., the discrete log of H with respect to G is unknown) is required so that an adversary who learns a holder secret x_s cannot translate it into an opening of the lock key. A *discrete-log pair* is a pair (x, X) with $X = x\,H$ for some scalar $x \in \mathbb{Z}_q$. Given only X and H, no efficient algorithm can recover x; this is the *elliptic-curve discrete-logarithm problem* (ECDLP), on which the security of Bitcoin and of our proofs rests.

2.2 The Paillier Cryptosystem

The Paillier cryptosystem [48] is an asymmetric encryption scheme whose plaintext space is $\mathbb{Z}_N$ for an RSA modulus $N = pq'$ (we write q' to distinguish the Paillier factor from the curve order q). The public key is (N, g) with $g \in \mathbb{Z}^*_{N^2}$ of order divisible by N. Encryption of a message $m \in \mathbb{Z}_N$ with randomness $\rho \in \mathbb{Z}^*_N$ is

$$\mathrm{Enc}_{pk}(m;\rho) = g^m \rho^N \bmod N^2.$$

Decryption uses the factorization of N. The scheme is semantically secure under the *Decisional Composite Residuosity* (DCR) assumption.

Additive homomorphism. Paillier is *additively* homomorphic: given ciphertexts $c_1 = \mathrm{Enc}(m_1)$ and $c_2 = \mathrm{Enc}(m_2)$, the product

$$c_1 \cdot c_2 \bmod N^2 = \mathrm{Enc}(m_1 + m_2)$$

is a valid encryption of $m_1 + m_2 \bmod N$. We write this operation as $c_1 \oplus c_2$, and the corresponding "ciphertext subtraction" as $c_1 \ominus c_2 := c_1 \cdot c_2^{-1} \bmod N^2$. This additive homomorphism is the reason Thunderbolt can compose a fresh holder ciphertext without decrypting the invoice ciphertext.

2.3 Sigma Protocols and Fiat–Shamir

A *Sigma protocol* [21, 22] is a three-move honest-verifier zero-knowledge proof: the prover sends commitment A, receives challenge c, responds with z; the verifier checks a public equation on (A, c, z). The *Fiat–Shamir heuristic* [32] compiles it into a NIZK in the random-oracle model by setting $c = \mathsf{Hash}(\text{statement}, A)$; soundness follows from the forking lemma [50].

2.4 Verifiable Encryption of Discrete Logarithms

Camenisch and Shoup [16] give an efficient protocol for *verifiable encryption of discrete logarithms*: given a discrete-log pair $(m, M = mH)$ and a Paillier ciphertext $ct = \mathrm{Enc}_{pk}(m)$, the prover produces a Sigma-protocol proof that ct encrypts the same integer whose discrete log is M. The proof relation is

$$\mathcal{R}_{\text{VE}} = \{(M, ct;\ m, \rho) : M = mH \ \wedge\ ct = g^m \rho^N \bmod N^2\}.$$

The key technique is a *shared response*: the Sigma-protocol response z simultaneously satisfies both the elliptic-curve verification equation ($zH \stackrel{?}{=} A_{\text{EC}} + cM$) and the Paillier verification equation ($g^z \tau^N \stackrel{?}{=} A_{\text{Pai}} \cdot ct^c$), where c is the Fiat–Shamir challenge and the same z forces the same underlying integer in both relations. This "shared-z" construction is the technical core of our proofs.

2.5 Verifiable Encrypted Secret Sharing (VESS)

Shoup's *Simple VESS* [56] provides a mechanism for verifiably encrypting a Shamir secret sharing [55] to a committee of n shareholders, building on the line of work initiated by Feldman [31], Pedersen [49], and the distributed key-generation protocols of [35]. The sharing party (the "dealer") publishes encrypted shares $(\hat{s}_1, \ldots, \hat{s}_n)$ together with a proof that there exists a unique polynomial of degree at most $t-1$ whose evaluations are the plaintexts of these ciphertexts. Any observer can verify the consistency of the encrypted shares without decrypting them. If at least t of the n shareholders are honest, they can collectively reconstruct the shared secret.

In Thunderbolt, VESS is used when the Committee is instantiated as a threshold committee rather than a single actor. The committee-side refresh value r_s is shared via a VESS protocol so that the compensating update $y_{s+1} = y_s - r_s$ is performed collectively, without any single committee member learning y_s or r_s in the clear. This modular substitution replaces the single-committee

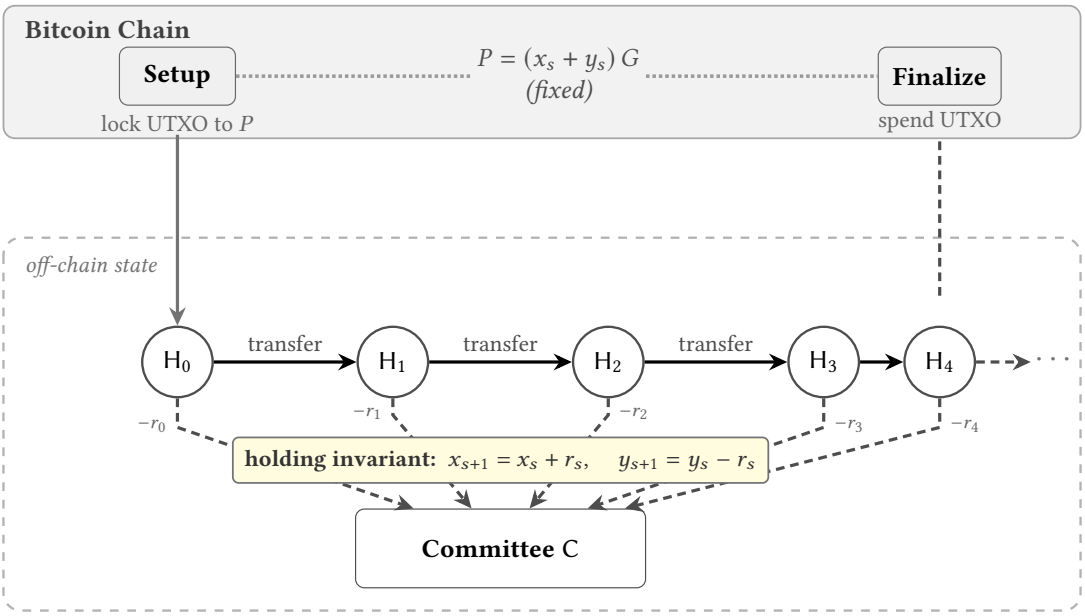


Figure 1: Illustrative overview of Thunderbolt. A Bitcoin UTXO is locked once on-chain under the fixed key $P = (x_s + y_s)G$ at Setup, and the chain sees no further activity until Finalize. Ownership transfers between successive holders $H_0, H_1, \ldots$ happen entirely off-chain (dashed box). At each step, a fresh refresh value r_s is added to the current holder's secret and subtracted from the Committee share, so that $(x_s + y_s)G$ remains equal to P throughout.

trust assumption with a standard threshold-honesty assumption, leaving the per-step NIZK proofs unchanged.

3 System Model and Design Rationale

A Thunderbolt channel involves three parties (a sequence of holders, a threshold committee, and the Bitcoin Chain) operating under an additive key-sharing invariant. We specify their roles and trust assumptions (Section 3.1), state the security goals (Section 3.2), and explain why the VEDL + VESS composition is both necessary and sufficient (Section 3.3).

3.1 Participants and Trust Assumptions

A Thunderbolt channel has three participant roles:

- The **Holder** (initially H_0) is the party that currently owns the UTXO off-chain. Over the lifetime of the channel, the holder role is handed from one user to the next: $H_0 \to H_1 \to \cdots$, where each H_s is an independent user.
- The **Committee** $C = \{C_1, \ldots, C_n\}$ is a fixed (but possibly threshold) group that maintains the complementary share of the lock key. C never owns the UTXO and never learns the holder secret; its sole role is to preserve the fixed-key invariant via the compensating update.
- The **Bitcoin Chain** plays the role of a passive verifier: it sees exactly two transactions over the channel's lifetime (Setup and Finalize), regardless of how many holder transitions occur in between.

We consider a static-corruption adversary $\mathcal{A}$ that may corrupt any subset of holders (past or future) and at most $t - 1$ of the n committee members before the channel opens. The adversary is probabilistic polynomial-time (PPT) and interacts with the protocol in the random-oracle model, which Section 5 instantiates as the $(\mathcal{F}_{\text{RO}}, \mathcal{F}_{\text{PKI}})$-hybrid. This trust model is identical to that of standard threshold-signing deployments (FROST [40], ROAST [53]).

3.2 Security Goals

Thunderbolt aims to provide three properties:

(1) **Ownership security:** Only the current holder, the party who knows the current holder secret x_s, can authorize a transfer or participate in channel finalization.
(2) **Fixed-key integrity:** The on-chain lock key P remains unchanged throughout the channel's lifetime, regardless of how many off-chain transfers occur.
(3) **Transfer correctness:** After a valid transfer from H_s to H_{s+1}, the new holder obtains a valid holder secret x_{s+1} and the committee's share y_{s+1} is updated consistently with the fixed-key invariant.

Thunderbolt provides no liveness guarantee (fewer than t responsive members freezes the channel) and does not hide the transfer graph.

3.3 Design Rationale

A correct transfer step must produce three cryptographic objects simultaneously: a Paillier ciphertext of $x_{s+1} = x_s + r_s$ under Bob's

key, a Paillier ciphertext of the refresh r_s under the committee's key, and a public anchor delta $D_s = r_s H$ on the elliptic curve. All three must be *bound to the same integer*; otherwise a dishonest party can break the fixed-key invariant $(x_s + y_s) G = P$.

VEDL for per-step correctness. Verifiable encryption of discrete logarithms [16] (Section 2.4) provides this binding. Its shared Sigma response z forces the Paillier plaintext and the elliptic-curve discrete log to be the same integer. We instantiate VEDL in two ways: the Transfer Consistency Proof uses a single-receiver instance; the Invoice Consistency Proof extends VEDL to a multi-receiver setting with a single shared z binding three objects simultaneously.

VESS for threshold trust. VEDL alone concentrates the committee role in a single actor. VESS [56] (Section 2.5) removes this single point of failure: the refresh r_s is verifiably encrypted as a (t, n)-Shamir sharing, so that any t honest members can perform $y_{s+1} = y_s - r_s$ collectively, while no coalition of fewer than t learns anything. This substitution is modular: VESS replaces the single ciphertext ct_C with a set of verifiably encrypted shares, and neither proof changes form.

Necessity of the composition. Neither primitive alone suffices. VEDL without VESS forces a single trusted third party. VESS without VEDL cannot bind the shared secret to the on-chain elliptic-curve anchor. Together, VEDL enforces per-step correctness and VESS distributes the committee role across a threshold group.

4 Construction

The VEDL + VESS composition identified in Section 3.3 is realized by two NIZK proofs. The Invoice Consistency Proof (Section 4.2) binds the three per-step objects to a single integer; the Transfer Consistency Proof (Section 4.3) certifies that the sender holds the current owner credential. Section 4.5 extends the single-committee construction to a threshold group via Simple VESS, and Section 4.6 assembles the channel lifecycle.

Throughout, we use the notation of Section 2.1: $\mathbb{G}$ is secp256k1 with prime order q, standard base point G (used for the lock key P), and NUMS generator H (used for all anchors and deltas). Let (N_B, g_B) and (N_C, g_C) be Paillier public keys for Bob and the Committee, respectively. All Paillier arithmetic is modulo N_B^2 or N_C^2 as appropriate. All elliptic-curve scalar equalities are taken modulo q. Let κ denote the statistical security parameter (concretely, $\kappa = 128$); we set $\tilde{q} = q^2 \cdot 2^\kappa$ as the masking range for Sigma-protocol commitment scalars, which ensures statistical zero-knowledge with distance at most $2^{-\kappa}$.

4.1 Transfer Model

A Thunderbolt state at step s consists of a *holder secret* x_s, a *committee secret* y_s, and a *public anchor* $X_s = x_s H$, subject to the fixed-key invariant

$$P = (x_s + y_s) G. \tag{1}$$

Alice is the current holder; Bob is the next recipient.

A transfer from Alice to Bob proceeds in two phases:

(1) **Invoice phase (Bob).** Bob chooses a fresh refresh value $r_s \in \mathbb{Z}_q$ and publishes:
- a public delta point $D_s = r_s H$,
- a ciphertext to himself, $\mathrm{ct}_B = \mathrm{Enc}_{pk_B}(r_s;\, \rho_B)$,
- a ciphertext to the Committee, $\mathrm{ct}_C = \mathrm{Enc}_{pk_C}(r_s;\, \rho_C)$,
- a proof π_{inv} (the Invoice Consistency Proof) that all three objects encode the same r_s.

(2) **Consume phase (Alice).** Alice forms the next holder ciphertext

$$T_{s+1} = \mathrm{Enc}_{pk_B}(x_s;\, \eta_s) \oplus \mathrm{ct}_B,$$

updates the public anchor to $X_{s+1} = X_s + D_s$, and publishes a proof π_{xfer} (the Transfer Consistency Proof) that she injected the actual current holder secret x_s consistent with X_s.

After the transfer, Bob decrypts T_{s+1} to obtain $x_s + r_s$ and sets $x_{s+1} \equiv x_s + r_s \pmod q$. The Committee recovers r_s from ct_C and updates $y_{s+1} = y_s - r_s$. The fixed-key invariant (1) is preserved:

$$(x_{s+1} + y_{s+1})\, G = (x_s + r_s + y_s - r_s)\, G = (x_s + y_s)\, G = P.$$

Security assumptions. Both proofs rely on ECDLP hardness in $\mathbb{G}$, the DCR assumption [48] for the Paillier moduli, and the random-oracle model [8] for Fiat–Shamir. Under these, both are complete, specially sound, and HVZK as Sigma protocols, and NIZK after Fiat–Shamir; soundness follows from the forking lemma [50].

4.2 Invoice Consistency Proof

Definition 4.1 (Invoice Consistency Relation). *The relation* $\mathcal{R}_{\mathrm{inv}}$ *is defined as*

$$\mathcal{R}_{\mathrm{inv}} = \left\{ (D_s, \mathrm{ct}_B, \mathrm{ct}_C;\ r_s, \rho_B, \rho_C) \ :\ \begin{array}{l} D_s = r_s H, \\ \mathrm{ct}_B = g_B^{r_s}\, \rho_B^{N_B} \bmod N_B^2, \\ \mathrm{ct}_C = g_C^{r_s}\, \rho_C^{N_C} \bmod N_C^2 \end{array} \right\}.$$

Security motivation. Without this proof, a malicious Bob could commit different refresh values in D_s, ct_B, and ct_C, breaking the fixed-key invariant.

Sigma protocol. The prover (Bob) samples a commitment scalar $\alpha \leftarrow \mathbb{Z}_{\tilde{q}}$ and commitment randomnesses $\beta_B \leftarrow \mathbb{Z}_{N_B}^*$, $\beta_C \leftarrow \mathbb{Z}_{N_C}^*$, and computes:

$$A_D = \alpha H,$$
$$A_B = g_B^{\alpha}\, \beta_B^{N_B} \bmod N_B^2,$$
$$A_C = g_C^{\alpha}\, \beta_C^{N_C} \bmod N_C^2.$$

Fiat–Shamir challenge. The challenge is computed as

$$c = \mathrm{Hash}\big(\texttt{tb/invoice},\ \texttt{ctx_id},\ \texttt{inv_id},\ H,\ pk_B,\ pk_C,\ D_s,\ \mathrm{ct}_B,\ \mathrm{ct}_C,\ A_D,\ A_B,\ A_C\big) \bmod q.$$

Response. The prover computes a single shared response and two Paillier randomness responses:

$$z = \alpha + c \cdot r_s,$$
$$\tau_B = \beta_B \cdot \rho_B^c \bmod N_B,$$
$$\tau_C = \beta_C \cdot \rho_C^c \bmod N_C.$$

The proof is $\pi_{\mathrm{inv}} = (A_D, A_B, A_C, z, \tau_B, \tau_C)$.

Verification. The verifier accepts iff all three of the following hold (same z in all three):

(1) **EC check:** $z H \stackrel{?}{=} A_D + c \cdot D_s$.
(2) **Bob–Paillier check:** $g_B^z\, \tau_B^{N_B} \stackrel{?}{=} A_B \cdot \mathrm{ct}_B^c \pmod{N_B^2}$.
(3) **Committee–Paillier check:** $g_C^z\, \tau_C^{N_C} \stackrel{?}{=} A_C \cdot \mathrm{ct}_C^c \pmod{N_C^2}$.

Theorem 4.2 (Soundness of the Invoice Consistency Proof). *The Invoice Consistency Proof is a NIZK for $\mathcal{R}_{\text{inv}}$ in the random-oracle model. In particular, the shared response z in all three verification equations forces a single integer r_s to be the discrete log of D_s and the Paillier plaintext of both ct_B and ct_C.*

Proof. We establish the three standard properties.

Completeness. If the prover is honest, then $z = \alpha + c\, r_s$, $\tau_B = \beta_B \cdot \rho_B^c$, and $\tau_C = \beta_C \cdot \rho_C^c$. For the EC check: $z\,H = (\alpha + c\, r_s)\,H = \alpha\,H + c\, r_s\,H = A_D + c\,D_s$. For the Bob–Paillier check: $g_B^z\,\tau_B^{N_B} = g_B^{\alpha + c\,r_s}\,(\beta_B \cdot \rho_B^c)^{N_B} = (g_B^\alpha\,\beta_B^{N_B}) \cdot (g_B^{r_s}\,\rho_B^{N_B})^c = A_B \cdot \text{ct}_B^c$. The Committee–Paillier check is analogous.

Special soundness. Given two accepting transcripts $(A_D, A_B, A_C,\ \ c, z, \tau_B, \tau_C)$ and $(A_D, A_B, A_C,\ \ c', z', \tau_B', \tau_C')$ with $c \neq c'$, define $\Delta c = c - c'$ and $\Delta z = z - z'$ (computed as integers; since $\alpha \leftarrow \mathbb{Z}_{\tilde{q}}$ with $\tilde{q} = q^2 \cdot 2^\kappa$ and $c, c' \in \mathbb{Z}_q$, the response $z = \alpha + c\,r_s$ lies in $\mathbb{Z}_{\tilde{q}}$ without wrapping around). Because $\Delta z / \Delta c = r_s$ holds as an exact integer (the masking range ensures no modular cancellation), we extract: $r_s^* := \Delta z / \Delta c \in \mathbb{Z}$ (reducing modulo q gives the EC scalar satisfying $r_s^*\,H = D_s$). From the Bob–Paillier equation: $g_B^{\Delta z} \cdot (\tau_B/\tau_B')^{N_B} = \text{ct}_B^{\Delta c}$, from which we extract $\rho_B^* = (\tau_B/\tau_B')^{(\Delta c)^{-1}} \bmod N_B$ satisfying $\text{ct}_B = g_B^{r_s^*} \cdot (\rho_B^*)^{N_B}$ (the same integer r_s^* appears in the exponent, since the shared Δz drives both equations). The Committee–Paillier extraction is identical, yielding ρ_C^*. Because a single Δz drives all three extractions, the extracted witness r_s^* is the same integer in all three relations, establishing $(D_s, \text{ct}_B, \text{ct}_C;\ r_s^*, \rho_B^*, \rho_C^*) \in \mathcal{R}_{\text{inv}}$.

Honest-verifier zero-knowledge. The simulator $\mathcal{S}$, given the statement $(D_s, \text{ct}_B, \text{ct}_C)$, samples $z \leftarrow \mathbb{Z}_{\tilde{q}}$, $\tau_B \leftarrow \mathbb{Z}_{N_B}^*$, $\tau_C \leftarrow \mathbb{Z}_{N_C}^*$, programs the random oracle to return c, and sets: $A_D = z\,H - c\,D_s$, $A_B = g_B^z\,\tau_B^{N_B} \cdot \text{ct}_B^{-c} \bmod N_B^2$, $A_C = g_C^z\,\tau_C^{N_C} \cdot \text{ct}_C^{-c} \bmod N_C^2$. The simulated transcript $(A_D, A_B, A_C, c, z, \tau_B, \tau_C)$ is statistically indistinguishable from an honest transcript (statistical distance at most $2^{-\kappa}$), since the masking range $\tilde{q}$ absorbs the shift $c \cdot r_s$.

Soundness in the non-interactive (Fiat–Shamir compiled) setting follows from the forking lemma [50] applied in the random-oracle model. □

4.3 Transfer Consistency Proof

The Transfer Consistency Proof assumes that the Invoice Consistency Proof has been verified, so the public inputs D_s and ct_B are known to be consistent with a single r_s.

Definition 4.3 (Transfer Consistency Relation). *The relation $\mathcal{R}_{\text{xfer}}$ is defined as*

$$\mathcal{R}_{\text{xfer}} = \left\{ (X_s, \text{ct}_B, T_{s+1};\ x_s, \eta_s) \ :\ \begin{array}{l} X_s = x_s\,H, \\ T_{s+1} \cdot \text{ct}_B^{-1} = g_B^{x_s}\,\eta_s^{N_B} \bmod N_B^2 \end{array} \right\}.$$

Security motivation. Without this proof, Alice could inject an arbitrary $x_s' \neq x_s$ into T_{s+1}, leaving Bob with a ciphertext inconsistent with the public anchor.

Sigma protocol. The prover (Alice) samples a commitment scalar $\alpha \leftarrow \mathbb{Z}_{\tilde{q}}$ and a Paillier commitment randomness $\beta \leftarrow \mathbb{Z}_{N_B}^*$, and computes:

$$A_X = \alpha\,H,$$
$$A_T = g_B^\alpha\,\beta^{N_B} \bmod N_B^2.$$

Fiat–Shamir challenge.

$$c = \text{Hash}\big(\texttt{tb/xfer},\ \texttt{ctx_id},\ s{+}1,\ \texttt{inv_id},\ H,\ pk_B,\ X_s,\ \text{ct}_B,\ T_{s+1},\ A_X,\ A_T\big) \bmod q.$$

Response.

$$z = \alpha + c \cdot x_s,$$
$$\tau = \beta \cdot \eta_s^c \bmod N_B.$$

The proof is $\pi_{\text{xfer}} = (A_X, A_T, z, \tau)$.

Verification. The verifier accepts iff both of the following hold (same z):

(1) **EC check:** $z\,H \stackrel{?}{=} A_X + c \cdot X_s$.
(2) **Paillier check:** $g_B^z\,\tau^{N_B} \stackrel{?}{=} A_T \cdot (T_{s+1} \ominus \text{ct}_B)^c \pmod{N_B^2}$.

Additionally, the verifier checks the public anchor transition:

$$X_{s+1} \stackrel{?}{=} X_s + D_s.$$

This check is outside the zero-knowledge relation itself but is necessary for protocol correctness.

Theorem 4.4 (Soundness of the Transfer Consistency Proof). *The Transfer Consistency Proof is a NIZK for $\mathcal{R}_{\text{xfer}}$ in the random-oracle model. The shared response z forces a single integer x_s to be both the discrete log of X_s and the Paillier plaintext of $T_{s+1} \ominus \text{ct}_B$.*

Proof. Let $U = T_{s+1} \ominus \text{ct}_B = T_{s+1} \cdot \text{ct}_B^{-1} \bmod N_B^2$ for brevity.

Completeness. With $z = \alpha + c\,x_s$ and $\tau = \beta \cdot \eta_s^c$: $z\,H = \alpha\,H + c\,x_s\,H = A_X + c\,X_s$, and $g_B^z\,\tau^{N_B} = g_B^\alpha\,\beta^{N_B} \cdot (g_B^{x_s}\,\eta_s^{N_B})^c = A_T \cdot U^c$.

Special soundness. Given two accepting transcripts with challenges $c \neq c'$, define $\Delta c = c - c'$, $\Delta z = z - z'$ (as integers; the masking range $\tilde{q}$ ensures no wrap-around). We extract the witness as the exact integer $x_s^* := \Delta z / \Delta c \in \mathbb{Z}$ (reducing modulo q gives the EC scalar satisfying $X_s = x_s^*\,H$). From the Paillier equation: $\eta_s^* = (\tau/\tau')^{(\Delta c)^{-1}} \bmod N_B$ satisfying $U = g_B^{x_s^*}\,(\eta_s^*)^{N_B} \bmod N_B^2$ (the same integer x_s^* appears in the Paillier exponent because it is derived from the shared Δz). Together, $(X_s, U;\ x_s^*, \eta_s^*) \in \mathcal{R}_{\text{xfer}}$.

Honest-verifier zero-knowledge. The simulator samples $z \leftarrow \mathbb{Z}_{\tilde{q}}$, $\tau \leftarrow \mathbb{Z}_{N_B}^*$, programs c via the random oracle, and sets $A_X = z\,H - c\,X_s$ and $A_T = g_B^z\,\tau^{N_B} \cdot U^{-c} \bmod N_B^2$. The resulting transcript is statistically indistinguishable from an honest one (statistical distance at most $2^{-\kappa}$).

As with the Invoice proof, non-interactive soundness follows from the forking lemma [50]. □

4.4 Joint Consequence: Correct Transfer Step

Theorem 4.5 (Transfer Correctness). *If both π_{inv} and π_{xfer} verify, and the anchor transition $X_{s+1} = X_s + D_s$ holds, then there*

exist integers x_s and r_s such that

$$X_s = x_s\, H, \tag{2}$$
$$D_s = r_s\, H, \tag{3}$$
$$T_{s+1} = \mathrm{Enc}_{pk_B}(x_s + r_s), \tag{4}$$
$$X_{s+1} = (x_s + r_s)\, H. \tag{5}$$

Furthermore, the committee-side update $y_{s+1} = y_s - r_s$ preserves the invariant (1):

$$(x_{s+1} + y_{s+1})\, G = (x_s + y_s)\, G = P.$$

Proof. From the Invoice Consistency Proof (Theorem 4.2), extract r_s satisfying $D_s = r_s\, H$ and $\mathrm{ct}_B = \mathrm{Enc}_{pk_B}(r_s)$. From the Transfer Consistency Proof (Theorem 4.4), extract x_s satisfying $X_s = x_s\, H$ and $T_{s+1} \ominus \mathrm{ct}_B = \mathrm{Enc}_{pk_B}(x_s)$. By Paillier's additive homomorphism, $T_{s+1} = \mathrm{Enc}_{pk_B}(x_s) \oplus \mathrm{Enc}_{pk_B}(r_s) = \mathrm{Enc}_{pk_B}(x_s + r_s)$, giving (4). The anchor transition gives $X_{s+1} = X_s + D_s = x_s\, H + r_s\, H = (x_s + r_s)\, H$, giving (5). Setting $x_{s+1} \equiv x_s + r_s \pmod q$ and $y_{s+1} \equiv y_s - r_s \pmod q$, we obtain $(x_{s+1} + y_{s+1})\, G = (x_s + y_s)\, G = P$. □

4.5 Threshold Committee via Simple VESS

The construction presented so far assumes a *single* third party C that holds the committee share y_s and receives the refresh ciphertext $\mathrm{ct}_C = \mathrm{Enc}_{pk_C}(r_s)$. This is a strong trust assumption: a compromised C can collude with any holder to recover the full lock key $x_s + y_s$ and steal the UTXO. In this subsection we show how to replace C with a threshold committee of n members, of which any t suffice to perform the compensating update, while no coalition of fewer than t learns anything about y_s or r_s. The key observation is that this upgrade is a *modular substitution*: we replace the single Paillier ciphertext ct_C with a Simple VESS distribution [56], and the Invoice and Transfer Consistency Proofs remain unchanged.

Setup. Let the committee consist of n members $\mathsf{C}_1, \ldots, \mathsf{C}_n$, each holding a Paillier key pair (pk_i, sk_i). At channel open, the initial committee share y_0 is distributed via a (t, n)-Shamir secret sharing: the dealer samples a degree-$(t-1)$ polynomial g_0 with $g_0(0) = y_0$ and sends share $y_0^{(i)} = g_0(i)$ to C_i. The public verification key is the set of *polynomial coefficient commitments* $\{F_j = a_j\, G\}_{j=0}^{t-1}$, where $g_0(X) = \sum_{j=0}^{t-1} a_j X^j$; this is the standard Feldman [31] commitment scheme used by Simple VESS. Any observer verifies C_i's share by checking $y_0^{(i)}\, G \stackrel{?}{=} \sum_{j=0}^{t-1} i^j\, F_j$; any t consistent shares reconstruct y_0, and no set of fewer than t shares reveals any information about y_0.

Invoice phase modification. When Bob issues an invoice at step s, he replaces the single ciphertext ct_C with a VESS distribution of r_s: he samples a degree-$(t-1)$ polynomial f with $f(0) = r_s$, encrypts each share $r_s^{(i)} = f(i)$ under pk_i, publishes polynomial commitments $\{F_j = a_j\, H\}_{j=0}^{t-1}$, and attaches the VESS consistency proof that the encrypted shares are evaluations of f at points $1, \ldots, n$ [56].

Because there is no longer a single committee ciphertext, the Invoice Consistency Proof π_{inv} in the threshold setting covers only the two-receiver relation (D_s, ct_B): it proves that D_s and ct_B both encode the same r_s. The committee-side binding is handled separately by the VESS consistency equations, which guarantee that the encrypted shares correspond to a unique degree-$(t-1)$ polynomial whose free-term commitment is F_0. The bridge between the two proofs is the public identity $F_0 = D_s$: π_{inv} certifies that D_s commits to r_s, and the VESS proof certifies that the shares encode a polynomial with free term committed to $F_0 = D_s$. Together they force every committee member's share to be a consistent evaluation of the same r_s's sharing polynomial.

Committee update. Upon receiving the VESS distribution, each committee member C_i decrypts $\hat{r}_s^{(i)}$ to recover $r_s^{(i)}$ and updates its local share of the committee secret:

$$y_{s+1}^{(i)} = y_s^{(i)} - r_s^{(i)}.$$

This update is performed locally by each member without communication. No member learns r_s itself; each member only sees its own share $r_s^{(i)}$.

Reconstruction and finalization. When the current holder wishes to finalize the channel (i.e., spend the UTXO on-chain), the protocol proceeds in two stages. First, any t committee members reconstruct y_s via Lagrange interpolation on their shares $\{y_s^{(i)}\}_{i \in S}$, $|S| \geq t$; this is an internal committee operation whose secrecy is protected by the (t, n)-threshold honesty assumption. Second, the committee (holding y_s) and the holder (holding x_s) run an *additive two-party Schnorr* protocol: each party samples a fresh nonce (k_1 and k_2), publishes the corresponding nonce point ($R_1 = k_1 G$, $R_2 = k_2 G$), computes the combined challenge $e = \mathsf{Hash}(R_1 + R_2,\, P,\, m)$, and responds with its partial response ($s_1 = k_1 + e\, x_s$, $s_2 = k_2 + e\, y_s$); the signature $(R_1 + R_2,\ s_1 + s_2)$ is valid under $P = (x_s + y_s)\, G$ [41]. Neither party learns the other's key share: the holder sees only R_2 and s_2, from which y_s is not recoverable without k_2. This two-party structure differs from FROST [40], which distributes shares of a *single* key among all signers; here the key is additively split between two independent roles (holder and committee), and FROST is used only if the committee itself requires distributed signing among its n members.

Correctness. The share update preserves the Shamir structure. At step s, define the committee polynomial $g_s(\cdot)$ by $g_s(i) = y_s^{(i)}$. At step $s + 1$:

$$g_{s+1}(i) = y_s^{(i)} - r_s^{(i)} = g_s(i) - f(i),$$

where f is the degree-$(t-1)$ sharing polynomial for r_s. Since both g_s and f are degree-$(t-1)$ polynomials, $g_{s+1} = g_s - f$ is also degree-$(t-1)$, and its free term is $y_s - r_s = y_{s+1}$. Thus the shares $\{y_{s+1}^{(i)}\}$ form a valid (t, n)-Shamir sharing of y_{s+1}, and the fixed-key invariant $(x_{s+1} + y_{s+1})\, G = P$ is preserved.

Security. The threshold committee instantiation inherits its security from two independent guarantees:

- **Secrecy of y_s:** As long as fewer than t committee members are corrupted, the adversary obtains fewer than t shares of y_s and learns nothing about the committee secret (by the information-theoretic security of Shamir sharing).
- **Consistency of r_s:** The VESS verification equations ensure that a malicious Bob cannot distribute inconsistent shares; any observer can verify that the encrypted shares are consistent with a single degree-$(t-1)$ polynomial whose free term matches D_s. If the VESS verification passes but the shares are inconsistent, the observer detects this and rejects the invoice.

The resulting trust assumption is the standard (t, n)-threshold honesty assumption: at most $t-1$ of the n committee members may be corrupted. This is the same assumption used by every modern threshold signing deployment (e.g., FROST [40], ROAST [53]), and it replaces the single-point-of-failure assumption of the basic construction with a well-understood distributed trust model.

Theorem 4.6 (VESS Modularity). *Let Π_{single} denote the single-third-party Thunderbolt construction of Section 4.1–Section 4.4, and let Π_{thr} denote the threshold-committee instantiation of this subsection. Then Π_{thr} preserves every property of Π_{single} (fixed-key invariance, anchor correctness, ciphertext binding, and soundness of the Invoice and Transfer Consistency Proofs) under the assumption that at most $t-1$ of the n committee members are corrupted and that the Simple VESS distribution of [56] is secure.*

Proof. The Transfer Consistency Proof π_{xfer} depends only on D_s, ct_B, X_s, and T_{s+1}; none of these are affected by the VESS substitution, so π_{xfer} remains sound and zero-knowledge as before. In the threshold setting, π_{inv} is run on the two-receiver relation (D_s, ct_B), which is a strict sub-relation of $\mathcal{R}_{\text{inv}}$; its soundness and ZK properties follow by the same shared-z argument of Theorem 4.2. The committee-side binding is provided instead by the VESS consistency proof: VESS verification guarantees that the encrypted shares are evaluations of a unique degree-$(t-1)$ polynomial f with $f(0) = r_s$, where $r_s H = F_0 = D_s$ is the public free-term commitment. A malicious Bob who distributes inconsistent shares is detected by any observer, and a Bob who sets $F_0 \neq D_s$ is rejected by π_{inv}. Correctness of the committee-side update follows from the Shamir-preservation argument above. Secrecy of y_s and r_s holds information-theoretically as long as fewer than t members are corrupted. □

4.6 Channel Lifecycle

A Thunderbolt channel passes through three phases: **Open**, **Transfer** (zero or more), and **Finalize**.

Open (on-chain). The initial holder H_0 and the Committee derive $P = (x_0 + y_0)\,G$ and fund a Taproot UTXO [63] locked to P. The initial anchor is $X_0 = x_0 H$ and the initial holder ciphertext is $T_0 = \text{Enc}_{pk_{\text{H}_0}}(x_0)$.

Transfer. Each ownership transition follows the invoice-and-consume protocol of Section 4.1. Bob publishes $(D_s, \text{ct}_B, \text{ct}_C, \pi_{\text{inv}})$; Alice verifies π_{inv}, computes T_{s+1} and X_{s+1}, and publishes $(T_{s+1}, X_{s+1}, \pi_{\text{xfer}})$; the Committee verifies both π_{inv} and π_{xfer} against the current anchor X_s, then decrypts ct_C and updates $y_{s+1} = y_s - r_s$; Bob decrypts T_{s+1} to obtain x_{s+1}. No step requires Alice and Bob to be online simultaneously or touches the chain. Figure 2 illustrates the flow.

Finalize. The current holder and Committee jointly sign the closing transaction under P via a standard threshold Schnorr [40] or ECDSA [41] protocol.

By Theorem 4.5 and induction on s, the protocol preserves three invariants at every step: $(x_s + y_s)\,G = P$ (fixed key), $X_s = x_s H$ (anchor correctness), and $T_s = \text{Enc}_{pk_{\text{H}_s}}(x_s)$ (ciphertext correctness).

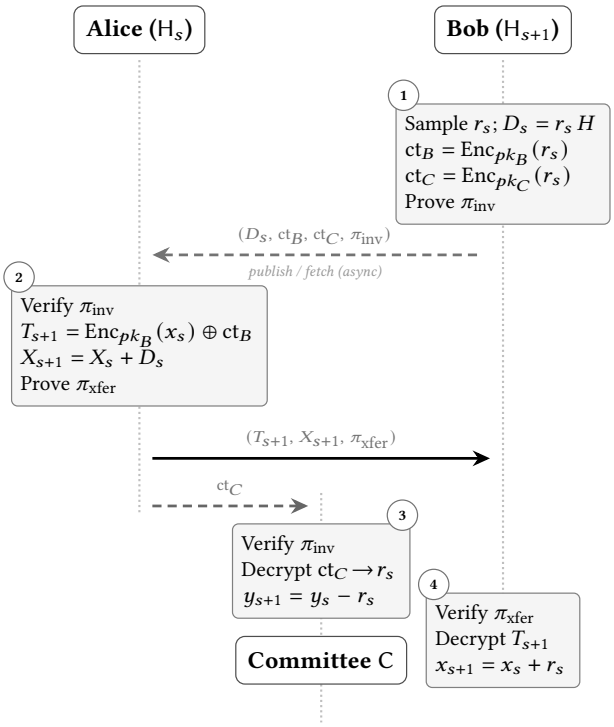


Figure 2: Transfer phase detail (one step $s \to s{+}1$). Bob creates an invoice with π_{inv} (①); Alice verifies π_{inv}, consumes the invoice, and publishes the updated ciphertext with π_{xfer} (②); the Committee verifies π_{inv} and applies the compensating subtract (③); Bob verifies π_{xfer} and decrypts the new holder secret (④). Steps ③ and ④ proceed independently.

5 Security Analysis

We formalize Thunderbolt's security in the UC framework [17]. A protocol *UC-realizes* $\mathcal{F}$ if every PPT real-world adversary can be simulated by an ideal-world simulator interacting only with $\mathcal{F}$, with computationally indistinguishable effect on any environment $\mathcal{Z}$; this implies security under arbitrary composition.

5.1 Hybrid Model

We analyze Thunderbolt in the $(\mathcal{F}_{\text{RO}}, \mathcal{F}_{\text{PKI}})$-hybrid model. $\mathcal{F}_{\text{RO}}$ is the standard random-oracle functionality [8], used by the Fiat–Shamir compilation of both NIZK proofs. $\mathcal{F}_{\text{PKI}}$ distributes Paillier public keys (pk_B, pk_C) to all parties before the channel opens. The threshold committee's Shamir sharing is information-theoretically secure and requires no additional ideal functionality. We consider a static-corruption adversary that may corrupt any subset of past and future holders and at most $t-1$ of the n committee members before the channel opens.

5.2 Ideal Functionality $\mathcal{F}_{\text{Thunderbolt}}$

$\mathcal{F}_{\text{Thunderbolt}}$ is parameterized by a prime-order group $\mathbb{G}$ and threshold (t, n). It interacts with an unbounded sequence of holder parties $\text{H}_0, \text{H}_1, \ldots$, committee members $\text{C}_1, \ldots, \text{C}_n$, the Bitcoin Chain, and the adversary $\mathcal{A}$. It maintains state $(s,\ \text{owner},\ P,\ \text{fin},\ \mathcal{P})$,

initialized to $(0,\ \mathsf{H}_0,\ \bot,\ \bot,\ \emptyset)$, where $\mathcal{P}$ is the set of pending (unconsumed) invoice identifiers.

Open. On (open, P) from H_0 and (open-ack) from at least t committee members: record $P \in \mathbb{G}$, set owner $\leftarrow \mathsf{H}_0$, and send (opened, P) to all parties.

Invoice. On (invoice) from any party B: sample a fresh identifier ι, add ι to $\mathcal{P}$, record that ι was issued to B, and send (invoice-ready, ι) to owner.

Transfer. On (transfer, ι) from owner, where ι was issued to recipient B:

- If fin $\neq \bot$ or $\iota \notin \mathcal{P}$, ignore.
- Remove ι from $\mathcal{P}$ (consuming it exactly once).
- Set $s \leftarrow s+1$ and owner $\leftarrow B$.
- Notify $\mathcal{A}$ of the state update.
- Deliver (ownership-received, s) to B when $\mathcal{A}$ schedules delivery, capturing B's asynchronous availability.

Finalize. On (finalize) from owner together with (finalize-ack) from at least t distinct committee members: set fin $\leftarrow \top$ and send (closing-tx, P, s) to the Bitcoin Chain.

$\mathcal{F}_{\text{Thunderbolt}}$ enforces by construction: (i) only the current owner can initiate a transfer; (ii) the on-chain key P is fixed at open time; (iii) each invoice identifier ι is consumed at most once, preventing equivocation; (iv) ownership delivery is deferred to adversarial scheduling, capturing the offline-recipient guarantee; and (v) finalization requires t honest committee members. These enforce *ownership security* via (i) and (v), *fixed-key integrity* via (ii), and *transfer correctness* plus the offline-recipient guarantee via (iii) and (iv).

5.3 UC Realization

Theorem 5.1 (Thunderbolt UC-realizes $\mathcal{F}_{\text{Thunderbolt}}$). *The Thunderbolt protocol UC-realizes $\mathcal{F}_{\text{Thunderbolt}}$ in the $(\mathcal{F}_{\text{RO}}, \mathcal{F}_{\text{PKI}})$-hybrid model, under the ECDLP assumption in $\mathbb{G}$ and the DCR assumption for the Paillier moduli, and assuming at most $t-1$ of the n committee members are statically corrupted.*

Proof sketch. We construct a simulator $\mathcal{S}$ and argue that no PPT environment $\mathcal{Z}$ can distinguish real from ideal execution with non-negligible probability.

Simulating proofs for honest senders. $\mathcal{S}$ holds no witness (x_s, η_s); it invokes the ZK simulators of Theorem 4.2 and Theorem 4.4 to produce transcripts $(\tilde{\pi}_{\text{inv}}, \tilde{\pi}_{\text{xfer}})$ statistically indistinguishable from real proofs, programming $\mathcal{F}_{\text{RO}}$ on fresh inputs.

Extracting from corrupted senders. If a corrupted party produces accepting proofs $(\pi_{\text{inv}},\ \pi_{\text{xfer}})$, $\mathcal{S}$ programs $\mathcal{F}_{\text{RO}}$ to return two distinct challenges for the same commitment, yielding two accepting transcripts, and runs the special-soundness extractor of Theorem 4.4, yielding x_s^* with $X_s = x_s^* H$. By the ECDLP assumption, $x_s^* = x_s$, confirming the party holds the current-owner credential; $\mathcal{S}$ then forwards (transfer, ι) to $\mathcal{F}_{\text{Thunderbolt}}$. A corrupted party unable to produce valid proofs is rejected by honest verifiers, and no transfer occurs in either world.

Simulating committee members. $\mathcal{S}$ runs the honest member algorithm with fresh random coins (indistinguishable). For fewer-than-t corrupted members, their Shamir shares are uniformly random by information-theoretic security; $\mathcal{S}$ programs them as uniform values consistent with the public verification polynomial.

Finalization. For an honest finalization, $\mathcal{S}$ reconstructs y_s from t honest shares, combines it with the extracted x_s, and produces a signature under P. An adversary attempting unilateral finalization must either compute a discrete log of P (hard by ECDLP) or recover y_s from fewer than t Shamir shares (impossible by information-theoretic secrecy).

In all cases the simulated and real transcripts are computationally indistinguishable, so $\mathcal{Z}$ cannot distinguish the two worlds with non-negligible advantage. □

5.4 Attack Vector Analysis

We discuss three concrete attack patterns and identify the mechanisms by which $\mathcal{F}_{\text{Thunderbolt}}$ rules them out.

Committee–holder collusion. If t committee members collude with the current holder H_s, they can reconstruct y_s and unilaterally spend the UTXO, bypassing the protocol entirely. This is an inherent limitation of the (t,n)-threshold trust model: Thunderbolt provides security *conditional on* $|\mathcal{I}| < t$, exactly as FROST [40] and ROAST [53] do for threshold signing.

More broadly, any *past* holder $\mathsf{H}_{s'}$ who retains their secret $x_{s'}$ is equally dangerous: since $(x_{s'} + y_{s'})\,G = P$ for every step s', a coalition of t committee members who can reconstruct $y_{s'}$ from their share history can combine it with $x_{s'}$ to produce a valid on-chain signature. To close this window, committee members must *securely delete* their refresh shares $r_s^{(i)}$ immediately after each step, preventing retrospective reconstruction of past committee secrets $y_{s'}$. Without this deletion discipline, the set of parties capable of mounting the attack grows by one at each transfer.

In practice, committee selection should follow established principles from threshold custody deployments: *diversity* across organizational and jurisdictional domains; *stake or reputation* via on-chain bonds with slashing for detectable misbehavior; and *rotation* via periodic re-election with VESS-based resharing (Section 4.5). For high-value UTXOs, a 3-of-5 or 5-of-9 committee drawn from reputable custodians provides a trust model comparable to existing Bitcoin multisig custody solutions [1].

Double-spending and equivocation. A malicious holder H_s might attempt to (a) consume the same invoice twice, (b) issue valid transfer proofs to two different recipients, or (c) finalize on-chain while a transfer is in flight. Attack (a) is prevented by the *monotonic anchor*: each transfer advances X_s to $X_{s+1} = X_s + D_s$; the committee accepts a proof only against the current anchor, so a replay against the same invoice fails. Attack (b) is prevented identically: the committee processes only the first valid transfer for step s and advances its state, rejecting any subsequent proof for the same step. Attack (c) is mitigated by the cooperative nature of finalization: the holder needs t committee co-signatures and the committee can refuse to sign while a transfer is pending. All three cases correspond to $\mathcal{F}_{\text{Thunderbolt}}$ ignoring duplicate or out-of-order inputs via the single-consumption rule on invoice identifiers.

Denial-of-service by the committee. If fewer than t committee members are responsive, the channel is frozen. Liveness is not a

protocol goal (Section 3.2). Practical mitigations include: *redundancy* ($n \gg t$, e.g., t=3, n=7); an *on-chain timeout escape hatch* where the funding UTXO includes a timelock branch allowing the holder to unilaterally sweep after a pre-agreed period (e.g., 30 days), mirroring Lightning's commitment timeout; and *committee monitoring* with heartbeats and VESS resharing to replace non-responsive members.

6 Evaluation

We address four research questions:

- **RQ1** (Section 6.1): How fast is a single off-chain transfer, and where is the time spent?
- **RQ2** (Section 6.2): Does per-transfer cost remain constant as the proof chain grows?
- **RQ3** (Section 6.3): How does the threshold-committee path scale with committee size and fault tolerance?
- **RQ4** (Section 6.4): How does Thunderbolt's Sigma-protocol construction compare with general-purpose SNARK backends?

Experimental setup. All benchmarks were run on a single machine with an Apple M3 Max processor (14-core, 10 performance + 4 efficiency), 36 GB unified memory, macOS 15.2, compiled with `rustc` 1.90.0 in release mode with LTO enabled. The Thunderbolt implementation uses the `rust-paillier` crate for Paillier operations with a 3072-bit modulus ($|N| = 3072$, so all Paillier arithmetic operates over 6144-bit integers in $\mathbb{Z}_{N^2}$) and the `k256` crate for secp256k1 elliptic-curve arithmetic. The single-committee microbenchmarks report the average, minimum, and maximum over 100 iterations with a 3-iteration warmup discarded. Bob's per-member VESS share proofs are generated in parallel using `rayon`; all other steps are sequential as they would be on a single machine. The threshold-committee benchmark executes the full per-transfer threshold path over 5 iterations.

6.1 RQ1: Single-Transfer Latency

How fast is a single Thunderbolt off-chain transfer, and where is the time spent?

A single off-chain transfer executes nine stages: invoice setup, invoice prove, invoice verify, transfer setup, transfer prove, anchor verify, transfer verify, Bob decrypt, and committee update. Table 1 reports measured wall-clock time for each stage.

Cost breakdown. The total off-chain transfer completes in ≈ 1.05 **s** on average. Table 2 groups the nine stages by functional role.

ZKP proving dominates at 46%: Invoice Prove (316 ms) needs three Paillier exponentiations over 6144-bit integers, Transfer Prove (165 ms) two; EC operations are $\approx 10^3\times$ faster and negligible. This Paillier-heavy profile explains the circuit-backend overhead (Section 6.4).

Proof size. π_{inv} totals ≈ 2.3 KB (three commitments, one shared response, two randomness responses at $|N| = 3072$); π_{xfer} totals ≈ 1.5 KB; combined ≈ 3.8 KB per step.

Stage	Avg (ms)	Min (ms)	Max (ms)
Invoice setup	121.4	118.4	145.4
Invoice prove	316.3	308.8	353.0
Invoice verify	201.4	197.1	237.4
Transfer setup	65.4	63.8	73.3
Transfer prove	165.3	161.1	240.7
Anchor verify	< 0.01	< 0.01	< 0.01
Transfer verify	113.1	110.3	121.2
Bob decrypt	31.2	30.1	36.2
Committee update	31.1	30.2	37.2
Total (off-chain)	**1045.1**	**1028.7**	**1117.7**

Table 1: Per-stage wall-clock time for a single Thunderbolt off-chain transfer (100 iterations, 3072-bit Paillier modulus, secp256k1).

Category	Time (ms)	Share (%)
ZKP proving (Invoice + Transfer)	481.6	46.1
ZKP verification (Invoice + Transfer)	314.5	30.1
Setup (Invoice + Transfer)	186.8	17.9
Auxiliary (anchor, decrypt, committee)	62.3	6.0
Total	**1045.2**	**100**

Table 2: Cost breakdown by functional category.

6.2 RQ2: Scalability Over Many Transfers

Does per-transfer cost remain constant as the channel's proof chain grows?

We execute a single-committee channel for $m \in \{1, 10, 50, 100, 500, 1000\}$ sequential transfers and measure every transfer's wall-clock latency.

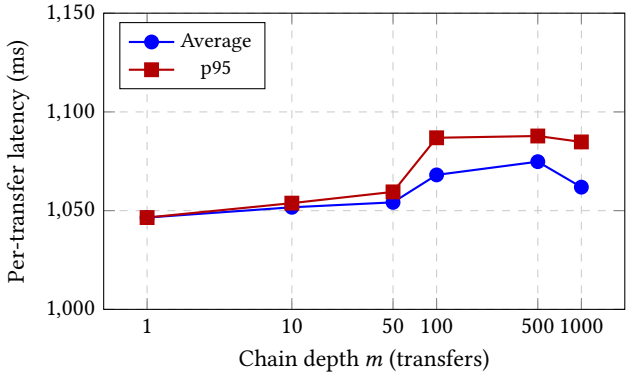


Figure 3: Per-transfer latency vs. chain depth for sequential transfers in a single-committee channel. Both average and p95 remain flat, confirming $O(1)$ per-transfer cost.

Per-transfer cost is constant. Figure 3 and Table 3 confirm that the m-th transfer takes the same ≈ 1.05–1.08 s as the first. Each transfer depends only on the current state (X_s, T_s, y_s); no party re-verifies

m	Avg (ms)	p50 (ms)	p95 (ms)	Total (s)	Proofs (KB)
1	1046.5	1046.5	1046.5	1.0	3.8
10	1051.7	1050.3	1053.8	10.5	38.0
50	1054.2	1053.0	1059.5	52.7	190.2
100	1068.1	1071.0	1086.9	106.8	380.4
500	1074.8	1072.4	1087.8	537.4	1901.9
1000	1061.9	1059.5	1084.8	1061.9	3803.7

Table 3: Scalability benchmark: per-transfer cost and cumulative proof storage for chains of length m.

the full proof chain. The p95 latency stays below 1.09 s at all chain depths.

Linear state growth. Storage grows at 3.8 KB/step (380 KB at m=100, 3.7 MB at m=1000); the full chain is needed only for dispute resolution, and older proofs can be discarded after the committee signs a checkpoint on (X_s, T_s).

6.3 RQ3: Threshold Committee Scalability

How does the threshold-committee transfer path scale with committee size and fault tolerance?

We benchmark the full threshold-committee transfer path of Section 4.5. The threshold is $t = \lceil 2n/3 \rceil$; we evaluate three committee sizes (4-of-6, 8-of-12, 12-of-18) and three malicious-member fractions (0%, 16.7%, 33.3%). Malicious members are modeled as non-participating during the committee-update phase. Bob generates all per-member VESS proofs in parallel (one CPU thread per member).

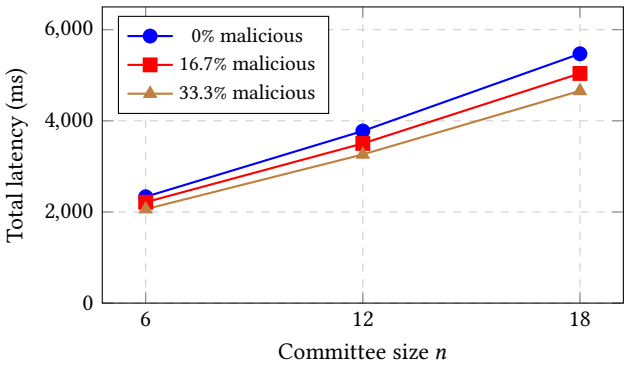


Figure 4: End-to-end threshold transfer latency for the full implemented threshold path, excluding one-time committee setup (5 iterations, release mode, parallel VESS generation).

Scaling with committee size. The total threshold-transfer latency scales roughly linearly with n: from 2.33 s at $n = 6$ to 5.47 s at $n = 18$ (Figure 4). The per-member marginal cost is ≈ 262 ms, dominated by sequential distribution verification and the benchmark's sequential committee-apply loop.

Effect of malicious fraction. For fixed n, the three malicious-fraction curves are close (Figure 4), with the 33.3% malicious curve

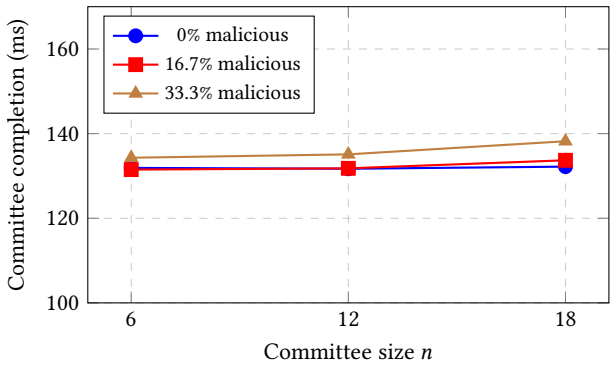


Figure 5: Committee-completion latency: time until the t-th honest member finishes decrypting and applying its refresh share (5 iterations, release mode).

faster than the 0% curve. This counterintuitive result arises because malicious members are absent from the committee-apply loop: with fewer participating members, the sequential apply total is shorter. Bob's upfront VESS generation (which depends on n, not on the honest count) is unaffected.

Under Simple VESS's (t, n)-honesty assumption, active corruption of the apply phase is equivalent to non-participation: a malicious member can silently corrupt its own local share, but as long as t honest members apply correctly, reconstruction succeeds using only those honest shares. The malicious member's corrupted share is simply excluded from the Lagrange interpolation subset. Benchmarking the absent-member scenario therefore covers the worst-case performance impact that any coalition of fewer than t corrupted members can cause.

Committee-side critical path. The committee-completion latency stays near 131–138 ms across all configurations (Figure 5) and is nearly independent of n. Each honest member performs a single share decryption and local update independently; the critical path is the t-th honest completion. In a real deployment, committee members work in parallel on separate machines, so the ≈ 132 ms per-member latency is the true committee bottleneck, not the sequential apply total reported in the stage breakdown.

Stage breakdown. Table 4 decomposes the threshold-transfer latency for the 0% malicious case. With parallel VESS generation, the VESS distribution stage is no longer the bottleneck: it takes 444 ms at $n = 6$ (previously 1520 ms sequentially). The benchmark's sequential distribution-verification total (709–1915 ms) is the largest reported stage, but in practice each committee member verifies only its own share (≈ 118 ms), so the real-world per-transfer overhead is again close to a single member's latency.

Stage	$n=6$ (ms)	$n=12$ (ms)	$n=18$ (ms)
VESS distribution (parallel)	444.5	485.9	747.7
Distribution verify (seq.)	709.6	1310.6	1915.7
Transfer prove+verify	343.4	342.0	341.2
Bob decrypt	31.1	31.0	31.1
Committee apply (seq. total)	791.2	1580.1	2394.9
Reconstruction	14.0	27.3	41.0
Total	**2333.8**	**3776.9**	**5471.6**

Table 4: Stage breakdown of threshold-transfer latency (0% malicious, 5 iterations). "Sequential" stages are parallelizable in a real deployment; the real-world critical-path latency per stage is ≈118 ms (verify) and ≈132 ms (apply).

6.4 RQ4: Comparison with SNARK Backends

How does Thunderbolt's Sigma-protocol construction compare with general-purpose SNARK backends on the same relations?

Sigma protocols vs. general-purpose SNARKs. An alternative realization would encode our relations as arithmetic circuits for a SNARK backend (Groth16 [36], Plonk [33], or a zkVM such as SP1 [59] or RISC Zero [52]). The Invoice and Transfer relations are dominated by Paillier modular exponentiation in $\mathbb{Z}_{N^2}$ for a 3072-bit N, which in a circuit requires bit-decomposition of 6144-bit integers and thousands of field operations per exponentiation. Each Paillier ciphertext verification alone consumes 10^5–10^6 R1CS constraints; EC scalar multiplication adds 10^3–10^4 more, placing the Invoice proof above 10^6 constraints in total. At published Groth16 proving rates of 10^4–10^5 constraints/s on commodity hardware, this yields a projected 60–300 s proving time, 125–625× slower than our 482 ms native prove. Even a simplified SP1 zkVM encoding of the same transfer relation on u64 arithmetic takes ∼8 s to prove (∼17× overhead), with the full 6144-bit version expected to be substantially worse.

Our direct Sigma-protocol construction requires no circuit: each proof consists of a constant number of native group operations. Concretely, proving both proofs takes ≈482 ms and verifying both takes ≈315 ms, with no trusted setup and no circuit-level tooling. Table 5 summarizes the comparison.

Backend	Prove	Verify	Slowdown	Setup
Thunderbolt (Sigma)	482 ms	315 ms	1×	None
SP1 zkVM*	∼8 s	∼1.2 s	∼17×	None
Groth16*	60–300 s	∼3 ms	125–625×	Trusted

Table 5: Prove/verify cost per transfer. *Conservative lower bounds; measured results will replace these estimates in the final version.

Groth16 proofs are smaller (288 bytes vs. our ≈3.8 KB) but require a per-circuit trusted setup and a projected 125–625× proving overhead. Our relations are algebraic (DL + Paillier equalities with a shared witness); Sigma protocols handle such relations natively, without circuit encoding. Thunderbolt trades a modestly larger proof for orders-of-magnitude lower proving cost and no trusted setup.

6.5 Limitations

Our benchmarks measure cryptographic computation on a single machine; a deployed channel involves network communication with variable RTT and geographic distribution. In a threshold setting, the committee-apply stage executes on separate machines: for a committee of $n = 6$ with 50 ms average RTT, network overhead adds ≈ 100 ms (one round-trip for share distribution and one for acknowledgement), a ∼ 4% increase over the 2.3 s computational cost for the parallel execution path. The SNARK numbers in Table 5 are conservative lower bounds (SP1 uses simplified u64 arithmetic; Groth16 is projected from published constraint rates); measured results on full relations will appear in the final version. Finally, statechains [57] share Thunderbolt's goal of off-chain UTXO transfer; a direct performance comparison is left to future work.

7 Related Work

Payment channels and layer-2 protocols. Off-chain payment channels date to Spilman [58] and were generalized by the Lightning Network [51], duplex channels [26], and eltoo [25]. Extensions address privacy and concurrency [43], atomic multi-channel updates [28], rebalancing [39], generalized state-channel machines [45], and miner-rationality defenses [5]; Gudgeon et al. [37] and Thyagarajan and Malavolta [62] survey the design space. Virtual channels [3, 6, 27] reduce on-chain cost by layering logical channels over existing funded ones, and channel factories [14] amortize opening transactions across many participants. Sleepy Channels [7] remove the watchtower requirement for bidirectional channels. Despite these advances, all designs fix the channel endpoints at open time. Thunderbolt removes this constraint by allowing the holder role to migrate across users without closing and re-opening.

Positioning relative to Lightning. Lightning optimizes for high-fan-out routing across pre-funded, bidirectionally-online channels with fixed endpoints and directional liquidity. Thunderbolt targets sequential ownership migration of a single UTXO with no routing, no directional capacity, and no simultaneous-online requirement. Both use exactly two on-chain transactions; Thunderbolt adds a (t, n)-threshold committee, trading Lightning's watchtower model for asynchronous, route-free operation.

Statechains. Somsen's statechain proposal [57] is the closest prior work: it transfers entire UTXOs off-chain via a trusted "statechain entity" that co-signs transactions. Thunderbolt differs in three respects: (1) the committee never signs off-chain transfer messages; it only maintains a secret share, while all validity is checked via public ZK proofs; (2) the fixed-key invariant shifts the key-deletion burden: whereas statechains require the statechain entity to provably destroy its old co-signing key at each transfer, Thunderbolt requires committee members to securely delete their per-step refresh shares $r_s^{(i)}$ after each update (see Section 5.4); unlike the statechain entity's unilateral deletion, committee-side deletion is enforceable by the threshold honesty assumption, since any t honest members can independently confirm that no further reconstructions of $y_{s'}$ are

possible once their shares are purged; (3) threshold-committee instantiation distributes trust across n members, whereas statechains concentrate it in a single co-signer. Mercury Layer [20] extends statechains with blinded signing to improve operator privacy but retains the single-entity trust model and the key-deletion assumption.

Adaptor signatures. Adaptor signatures [4, 29] condition a presignature on the discrete log of a public point, used in PTLC-based routing and atomic swaps. Thyagarajan et al. [30] formalize their security for general NP relations, and Tairi et al. [60] give a composable security framework for adaptor-signature-based blockchain protocols. Thunderbolt's verifiable encryption serves a related but distinct purpose: we encrypt the discrete log itself and prove consistency. The off-chain transfer mechanism (the core contribution) relies on Paillier homomorphism and VEDL proofs, not on signature adaptors.

Verifiable encryption and secret sharing. Camenisch and Shoup [16] give a Sigma-protocol proof that a Paillier ciphertext encrypts the discrete log of a group element, now standard in threshold cryptography, verifiable escrow [15], and optimistic fair exchange [2]. Ganesh et al. [34] give a UC-secure NIZK compiler from Sigma protocols, showing the composability potential of this proof family. Our Invoice and Transfer Consistency Proofs are direct instantiations of the Camenisch–Shoup technique, with the Invoice proof extending it to bind three objects (one EC point and two Paillier ciphertexts) via a single shared Sigma response. Shoup's Simple VESS [56] provides verifiable encryption of Shamir sharing [55], building on Feldman's VSS [31] and Pedersen commitments [49]. Das and Ren [23] recently simplify VSS with reduced bandwidth and latency. Thunderbolt uses VESS to distribute the committee-side refresh r_s under a (t, n)-threshold group at each transfer step.

Threshold signatures for Bitcoin. FROST [40] and ROAST [53] provide efficient threshold Schnorr [54] signing; Chu et al. [19] prove FROST secure without the algebraic group model. Das et al. [24] give practical asynchronous distributed key generation, and Thyagarajan et al. [61] address threshold-service payments on Bitcoin. These protocols can finalize a Thunderbolt channel when the on-chain output is a Taproot [64] key. Lindell [41] gives a two-party ECDSA protocol, and Canetti et al. [18] a proactive threshold-ECDSA protocol with UC security. Compact multi-signatures [11] offer a complementary route to reducing on-chain key material. These threshold signing protocols are complementary to Thunderbolt: they handle the on-chain finalization (the Finalize phase), while our Sigma-protocol proofs handle the off-chain state transitions.

General-purpose zero-knowledge proofs. Groth16 [36], Plonk [33], transparent STARKs [9], and zkVM-based systems such as Succinct SP1 [59] and RISC Zero [52] can in principle prove any NP statement. While powerful, these systems pay a large constant for algebraic relations that Sigma protocols handle natively. Recursive proof composition [12] and accumulation-based aggregation [10] reduce verification costs but do not eliminate proving-time overhead for Paillier-heavy statements.

Off-chain protocols beyond Bitcoin. Ethereum's layer-2 landscape includes optimistic rollups [47], zk-rollups [44], state channels [27, 45], and commit-chains [38]. These systems leverage Ethereum's expressive smart-contract layer to enforce state transitions on-chain when needed, an option unavailable in Bitcoin's restricted scripting model. In the Bitcoin context, Ark [13] proposes virtual UTXOs managed by a service provider who periodically settles on-chain, and BitVM [42] enables general computation verification via fraud proofs. Thunderbolt occupies a different point in the design space: it targets single-UTXO ownership transfer using only standard Taproot, requiring no new opcodes, no fraud-proof infrastructure, and no expressive scripting beyond what Bitcoin already provides.

8 Conclusion

We presented Thunderbolt, an off-chain Bitcoin protocol that transfers UTXO ownership asynchronously and recursively under a fixed on-chain lock key. The construction pairs two well-studied primitives (Camenisch–Shoup verifiable encryption and Simple VESS) in a new way: two Sigma-protocol NIZKs enforce the "one add, one subtract" invariant per step, while VESS distributes the committee share under a standard (t, n)-threshold-honesty assumption. A single transfer completes in ≈ 1 s with 3.8 KB proof size; per-transfer cost is $O(1)$ in chain length; the threshold path adds ≈ 262 ms per committee member with a committee-side critical path below 140 ms.

Complementary to Lightning's routed payment-channel network, Thunderbolt provides asynchronous, route-free ownership transfer: a transfer requires only a one-time invoice from the recipient, and the sender can consume it at any later time without coordination. This makes Thunderbolt well-suited for payroll batches, cold-wallet receipts, custody chains, and low-power devices that wake intermittently.

Thunderbolt introduces no new cryptographic assumption; both primitives have been scrutinized for over two decades. The contribution is a new *usage scenario* for these building blocks: asynchronous, recursive, off-chain Bitcoin ownership transfer under a fixed on-chain footprint.

Future directions. Several avenues remain open. First, a concrete analysis of specific composed systems (atomic cross-chain swaps, multi-channel constructions) would characterize the composed ideal functionalities and their interaction with Thunderbolt's trust assumptions. Second, end-to-end validation with geographically distributed committee members over real network links is needed; committee-completion results (Section 6.3) suggest the bottleneck will remain Bob's per-member VESS distribution. Third, the state-compaction mechanism (committee-signed checkpoints) requires a concrete protocol design, including the conditions under which a checkpoint safely replaces the full proof chain and how refresh-share deletion (Section 5.4) interacts with the checkpointing policy. Finally, a routed graph of Thunderbolt channels could combine Lightning's multi-hop reach with Thunderbolt's asynchronous, route-free endpoints.

References

[1] Andreas M. Antonopoulos. 2017. *Mastering Bitcoin: Programming the Open Blockchain* (2nd ed.). O'Reilly Media.

[2] N. Asokan, Victor Shoup, and Michael Waidner. 1998. Optimistic Fair Exchange of Digital Signatures. In *Advances in Cryptology — EUROCRYPT '98 (LNCS, Vol. 1403)*. Springer, 591–606. doi:10.1007/BFb0054156

[3] Lukas Aumayr, Oguzhan Ersoy, Andreas Erwig, Sebastian Faust, Kristina Hostáková, Matteo Maffei, Pedro Moreno-Sanchez, and Siavash Riahi. 2021. Bitcoin-Compatible Virtual Channels. In *2021 IEEE Symposium on Security and Privacy (S&P)*. IEEE, 901–918. doi:10.1109/SP40001.2021.00097

[4] Lukas Aumayr, Oguzhan Ersoy, Andreas Erwig, Sebastian Faust, Kristina Hostáková, Matteo Maffei, Pedro Moreno-Sanchez, and Siavash Riahi. 2021. Generalized Channels from Limited Blockchain Scripts and Adaptor Signatures. In *Advances in Cryptology — ASIACRYPT 2021 (LNCS, Vol. 13091)*. Springer, 635–666. doi:10.1007/978-3-030-92075-3_22

[5] Lukas Aumayr, Kasra Mazaheri, Matteo Maffei, and Pedro Moreno-Sanchez. 2024. Securing Lightning Channels against Rational Miners. In *Proceedings of the 2024 ACM SIGSAC Conference on Computer and Communications Security (CCS)*. ACM. doi:10.1145/3658644.3670373

[6] Lukas Aumayr, Pedro Moreno-Sanchez, Aniket Kate, and Matteo Maffei. 2023. Breaking and Fixing Virtual Channels: Domino Attack and Donner. In *30th Annual Network and Distributed System Security Symposium (NDSS)*. Internet Society.

[7] Lukas Aumayr, Sri AravindaKrishnan Thyagarajan, Giulio Malavolta, Pedro Moreno-Sanchez, and Matteo Maffei. 2022. Sleepy Channels: Bi-directional Payment Channels without Watchtowers. In *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security (CCS)*. ACM, 2521–2535. doi:10.1145/3548606.3559370

[8] Mihir Bellare and Phillip Rogaway. 1993. Random Oracles Are Practical: A Paradigm for Designing Efficient Protocols. In *Proceedings of the 1st ACM Conference on Computer and Communications Security (CCS)*. ACM, 62–73. doi:10.1145/168588.168596

[9] Eli Ben-Sasson, Iddo Bentov, Yinon Horesh, and Michael Riabzev. 2019. Scalable Zero Knowledge with No Trusted Setup. In *Advances in Cryptology — CRYPTO 2019 (LNCS, Vol. 11694)*. Springer, 701–732. doi:10.1007/978-3-030-26954-8_23

[10] Dan Boneh, Justin Drake, Ben Fisch, and Ariel Gabizon. 2020. Recursive Proof Composition from Accumulation Schemes. Cryptology ePrint Archive, Report 2020/499. https://eprint.iacr.org/2020/499

[11] Dan Boneh, Manu Drijvers, and Gregory Neven. 2018. Compact Multi-Signatures for Smaller Blockchains. In *Advances in Cryptology — ASIACRYPT 2018 (LNCS, Vol. 11273)*. Springer, 435–464. doi:10.1007/978-3-030-03329-3_15

[12] Sean Bowe, Jack Grigg, and Daira Hopwood. 2019. Halo: Recursive Proof Composition without a Trusted Setup. Cryptology ePrint Archive, Report 2019/1021. https://eprint.iacr.org/2019/1021

[13] Burak. 2023. Ark: A Second-Layer Protocol for Bitcoin. White paper. https://arkdev.info/

[14] Conrad Burchert, Christian Decker, and Roger Wattenhofer. 2018. Scalable Funding of Bitcoin Micropayment Channel Networks. In *Royal Society Open Science*, Vol. 5. doi:10.1098/rsos.180089

[15] Jan Camenisch and Anna Lysyanskaya. 2001. Efficient Non-transferable Anonymous Multi-show Credential System with Optional Anonymity Revocation. In *Advances in Cryptology — EUROCRYPT 2001 (LNCS, Vol. 2045)*. Springer, 93–118. doi:10.1007/3-540-44987-6_7

[16] Jan Camenisch and Victor Shoup. 2003. Practical Verifiable Encryption and Decryption of Discrete Logarithms. In *Advances in Cryptology — CRYPTO 2003 (LNCS, Vol. 2729)*. Springer, 126–144. doi:10.1007/978-3-540-45146-4_8

[17] Ran Canetti. 2001. Universally Composable Security: A New Paradigm for Cryptographic Protocols. In *42nd IEEE Symposium on Foundations of Computer Science (FOCS)*. IEEE, 136–145. doi:10.1109/SFCS.2001.959888

[18] Ran Canetti, Rosario Gennaro, Steven Goldfeder, Nikolaos Makriyannis, and Udi Peled. 2020. UC Non-Interactive, Proactive, Threshold ECDSA with Identifiable Aborts. In *Proceedings of the 2020 ACM SIGSAC Conference on Computer and Communications Security (CCS)*. ACM, 1769–1787. doi:10.1145/3372297.3423367

[19] Hien Chu, Paul Gerhart, Tim Ruffing, and Dominique Schröder. 2023. Practical Schnorr Threshold Signatures Without the Algebraic Group Model. In *Advances in Cryptology — CRYPTO 2023 (LNCS, Vol. 14081)*. Springer, 743–773. doi:10.1007/978-3-031-38557-5_24

[20] CommerceBlock. 2023. Mercury Layer: Statechain Implementation with Blinded Signing. Project documentation. https://mercurylayer.com/

[21] Ronald Cramer. 1997. *Modular Design of Secure yet Practical Cryptographic Protocols*. Ph.D. Thesis. University of Amsterdam.

[22] Ivan Damgård. 2010. On Σ-Protocols. Lecture notes, Aarhus University. https://www.cs.au.dk/~ivan/Sigma.pdf

[23] Sourav Das and Ling Ren. 2025. Verifiable Secret Sharing Simplified. In *2025 IEEE Symposium on Security and Privacy (S&P)*. IEEE.

[24] Sourav Das, Tom Yurek, Zhuolun Xiang, Andrew Miller, Lefteris Kober, and Ling Ren. 2022. Practical Asynchronous Distributed Key Generation. In *2022 IEEE Symposium on Security and Privacy (S&P)*. IEEE, 2518–2534. doi:10.1109/SP46214.2022.9833584

[25] Christian Decker, Rusty Russell, and Olaoluwa Osuntokun. 2018. eltoo: A Simple Layer2 Protocol for Bitcoin. White paper. https://blockstream.com/eltoo.pdf

[26] Christian Decker and Roger Wattenhofer. 2015. A Fast and Scalable Payment Network with Bitcoin Duplex Micropayment Channels. In *Stabilization, Safety, and Security of Distributed Systems (SSS) (LNCS, Vol. 9212)*. Springer, 3–18. doi:10.1007/978-3-319-21741-3_1

[27] Stefan Dziembowski, Sebastian Faust, and Kristina Hostáková. 2018. General State Channel Networks. In *Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security (CCS)*. ACM, 949–966. doi:10.1145/3243734.3243856

[28] Christoph Egger, Pedro Moreno-Sanchez, and Matteo Maffei. 2019. Atomic Multi-Channel Updates with Constant Collateral in Bitcoin-Compatible Payment-Channel Networks. In *Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security (CCS)*. ACM, 801–815. doi:10.1145/3319535.3345666

[29] Andreas Erwig, Sebastian Faust, Kristina Hostáková, Monosij Maitra, and Siavash Riahi. 2021. Two-Party Adaptor Signatures from Identification Schemes. In *Public-Key Cryptography — PKC 2021 (LNCS, Vol. 12710)*. Springer, 451–480. doi:10.1007/978-3-030-75245-3_17

[30] Andreas Erwig, Sebastian Faust, Kristina Hostáková, Monosij Maitra, and Sri AravindaKrishnan Thyagarajan. 2024. Foundations of Adaptor Signatures. In *Advances in Cryptology — EUROCRYPT 2024 (LNCS, Vol. 14654)*. Springer, 155–184. doi:10.1007/978-3-031-58723-8_6

[31] Paul Feldman. 1987. A Practical Scheme for Non-interactive Verifiable Secret Sharing. In *28th Annual Symposium on Foundations of Computer Science (FOCS)*. 427–438. doi:10.1109/SFCS.1987.4

[32] Amos Fiat and Adi Shamir. 1987. How to Prove Yourself: Practical Solutions to Identification and Signature Problems. In *Advances in Cryptology — CRYPTO '86 (LNCS, Vol. 263)*. Springer, 186–194. doi:10.1007/3-540-47721-7_12

[33] Ariel Gabizon, Zachary J. Williamson, and Oana Ciobotaru. 2019. PlonK: Permutations over Lagrange-bases for Oecumenical Noninteractive arguments of Knowledge. Cryptology ePrint Archive, Report 2019/953. https://eprint.iacr.org/2019/953

[34] Chaya Ganesh, Aayush Jain, Bernardo Magri, and Rafail Ostrovsky. 2025. Universally Composable NIZK from Sigma Protocols via a New Straight-line Compiler. In *Public-Key Cryptography — PKC 2025*. Springer.

[35] Rosario Gennaro, Stanisław Jarecki, Hugo Krawczyk, and Tal Rabin. 2007. Secure Distributed Key Generation for Discrete-Log Based Cryptosystems. *Journal of Cryptology* 20, 1 (2007), 51–83. doi:10.1007/s00145-006-0347-3

[36] Jens Groth. 2016. On the Size of Pairing-Based Non-interactive Arguments. In *Advances in Cryptology — EUROCRYPT 2016 (LNCS, Vol. 9666)*. Springer, 305–326. doi:10.1007/978-3-662-49896-5_11

[37] Lewis Gudgeon, Pedro Moreno-Sanchez, Stefanie Roos, Patrick McCorry, and Arthur Gervais. 2020. SoK: Layer-Two Blockchain Protocols. In *Financial Cryptography and Data Security (FC) (LNCS, Vol. 12059)*. Springer, 201–226. doi:10.1007/978-3-030-51280-4_12

[38] Harry Kalodner, Steven Goldfeder, Xiaoqi Chen, S. Matthew Weinberg, and Edward W. Felten. 2018. Arbitrum: Scalable, Private Smart Contracts. In *27th USENIX Security Symposium*. USENIX, 1353–1370.

[39] Rami Khalil and Arthur Gervais. 2017. Revive: Rebalancing Off-Blockchain Payment Networks. In *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security (CCS)*. ACM, 439–453. doi:10.1145/3133956.3134033

[40] Chelsea Komlo and Ian Goldberg. 2021. FROST: Flexible Round-Optimized Schnorr Threshold Signatures. In *Selected Areas in Cryptography (SAC) (LNCS, Vol. 12804)*. Springer, 34–65. doi:10.1007/978-3-030-81652-0_2

[41] Yehuda Lindell. 2017. Fast Secure Two-Party ECDSA Signing. In *Advances in Cryptology — CRYPTO 2017 (LNCS, Vol. 10402)*. Springer, 613–644. doi:10.1007/978-3-319-63715-0_21

[42] Robin Linus. 2023. BitVM: Compute Anything on Bitcoin. White paper. https://bitvm.org/bitvm.pdf

[43] Giulio Malavolta, Pedro Moreno-Sanchez, Aniket Kate, Matteo Maffei, and Srivatsan Ravi. 2017. Concurrency and Privacy with Payment-Channel Networks. In *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security (CCS)*. ACM, 455–471. doi:10.1145/3133956.3134096

[44] Matter Labs. 2023. zkSync: Scaling Freedom for All. Project documentation. https://zksync.io/

[45] Andrew Miller, Iddo Bentov, Surya Bakshi, Ranjit Kumaresan, and Patrick McCorry. 2019. Sprites and State Channels: Payment Networks that Go Faster than Lightning. In *Financial Cryptography and Data Security (FC) (LNCS, Vol. 11598)*. Springer, 508–526. doi:10.1007/978-3-030-32101-7_30

[46] Satoshi Nakamoto. 2008. Bitcoin: A Peer-to-Peer Electronic Cash System. White paper. https://bitcoin.org/bitcoin.pdf

[47] Offchain Labs. 2021. Arbitrum: Scalable, Private Smart Contracts. Project documentation. https://developer.arbitrum.io/

[48] Pascal Paillier. 1999. Public-Key Cryptosystems Based on Composite Degree Residuosity Classes. 1592 (1999), 223–238. doi:10.1007/3-540-48910-X_16

[49] Torben Pryds Pedersen. 1991. Non-Interactive and Information-Theoretic Secure Verifiable Secret Sharing. In *Advances in Cryptology — CRYPTO '91 (LNCS, Vol. 576)*. Springer, 129–140. doi:10.1007/3-540-46766-1_9

[50] David Pointcheval and Jacques Stern. 1996. Security Proofs for Signature Schemes. In *Advances in Cryptology — EUROCRYPT '96 (LNCS, Vol. 1070)*. Springer, 387–398. doi:10.1007/3-540-68339-9_33
[51] Joseph Poon and Thaddeus Dryja. 2016. The Bitcoin Lightning Network: Scalable Off-Chain Instant Payments. White paper. https://lightning.network/lightning-network-paper.pdf
[52] RISC Zero. 2024. RISC Zero: General-Purpose Verifiable Computing. Project documentation. https://www.risczero.com/
[53] Tim Ruffing, Viktoria Ronge, Elliott Jin, Jonas Schneider-Bensch, and Dominique Schröder. 2022. ROAST: Robust Asynchronous Schnorr Threshold Signatures. Cryptology ePrint Archive, Report 2022/550. https://eprint.iacr.org/2022/550
[54] Claus-Peter Schnorr. 1991. Efficient Signature Generation by Smart Cards. *Journal of Cryptology* 4, 3 (1991), 161–174. doi:10.1007/BF00196725
[55] Adi Shamir. 1979. How to Share a Secret. *Commun. ACM* 22, 11 (1979), 612–613. doi:10.1145/359168.359176
[56] Victor Shoup. 2024. Simple VESS (Verifiable Encrypted Secret Sharing). Cryptology ePrint Archive, Report 2024/253. https://eprint.iacr.org/2024/253
[57] Ruben Somsen. 2020. Statechains: Non-custodial Off-chain Bitcoin Transfer. White paper. https://github.com/RubenSomsen/rubensomsen.github.io/blob/master/img/statechains.pdf
[58] Jeremy Spilman. 2013. Anti DoS for tx replacement. Bitcoin-development mailing list. https://lists.linuxfoundation.org/pipermail/bitcoin-dev/2013-April/002433.html
[59] Succinct Labs. 2024. SP1: A zkVM for Developers. Project documentation. https://docs.succinct.xyz/
[60] Erkan Tairi, Pedro Moreno-Sanchez, and Matteo Maffei. 2023. LedgerLocks: A Security Framework for Blockchain Protocols Based on Adaptor Signatures. In *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security (CCS)*. ACM, 881–895. doi:10.1145/3576915.3623152
[61] Sri AravindaKrishnan Thyagarajan, Adithya Bhat, Giulio Malavolta, Nico Döttling, Aniket Kate, and Dominique Schröder. 2025. Vitarit: Paying for Threshold Services on Bitcoin and Friends. In *2025 IEEE Symposium on Security and Privacy (S&P)*. IEEE.
[62] Sri AravindaKrishnan Thyagarajan and Giulio Malavolta. 2025. Cryptographic Primitives in Script-based and Scriptless Payment Channel Networks: A Survey. *Comput. Surveys* (2025). doi:10.1145/3725846
[63] Pieter Wuille, Jonas Nick, and Tim Ruffing. 2020. BIP 340: Schnorr Signatures for secp256k1. Bitcoin Improvement Proposals. https://github.com/bitcoin/bips/blob/master/bip-0340.mediawiki
[64] Pieter Wuille, Jonas Nick, and Anthony Towns. 2020. BIP 341: Taproot: SegWit Version 1 Spending Rules. Bitcoin Improvement Proposals. https://github.com/bitcoin/bips/blob/master/bip-0341.mediawiki

Ethical Considerations

This work presents a cryptographic protocol for off-chain Bitcoin transfers. The protocol employs only standard primitives (Paillier encryption, Schnorr-based Sigma protocols, Shamir secret sharing) that have been publicly studied for over two decades. No user data was collected or processed; all experiments use synthetic cryptographic inputs on a locally controlled machine. The protocol does not introduce new mechanisms for bypassing existing blockchain compliance or anti-money-laundering frameworks; it operates within the same trust model as existing custody and multi-signature deployments. We see no direct ethical hazards beyond those already present in any privacy-enhancing cryptocurrency technology.

Open Science Policy